

Neural Networks and Backpropagation

A neural network is a function approximator built from layers of simple units called neurons. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a nonlinear activation function. Stacking many such layers lets the network represent highly complex relationships between inputs and outputs.

The activation function is what gives a network its expressive power. Without a nonlinearity, stacking layers would collapse into a single linear transformation.

Common choices include the sigmoid, which squashes values into the range zero to one; the hyperbolic tangent, centered at zero; and the rectified linear unit, or ReLU, which outputs zero for negative inputs and the input itself otherwise. ReLU is popular because it is cheap to compute and reduces the vanishing gradient problem.

Training a network means adjusting its weights so that its predictions match the desired outputs. This requires a loss function that measures how wrong the network is. For regression, mean squared error is typical; for classification, cross entropy loss is standard. The goal of training is to minimize the average loss over the training data.

Gradient descent is the optimization algorithm used to minimize the loss. It computes the gradient of the loss with respect to every weight, then nudges each weight a small step in the direction that reduces the loss. The size of that step is controlled by the learning rate. A learning rate that is too large causes the training to diverge; one that is too small makes training painfully slow.

Backpropagation is the algorithm that computes those gradients efficiently. It applies the chain rule of calculus, propagating the error backward from the output layer to the input layer, reusing intermediate results so that each gradient is computed only once. Backpropagation is what makes training deep networks computationally feasible.

Overfitting occurs when a network memorizes the training data instead of learning general patterns, causing poor performance on new data. Techniques such as regularization, dropout, and early stopping combat overfitting by discouraging the network from relying too heavily on any single feature or weight.